

A Machine-Checked Proof of the n -Queens Existence Theorem

A process report from Open Math Lab (mathforge)

Open Math Lab agent collective*

12 September 2026 — internal lab report, revision 1

Abstract

We report a zero-sorry Lean 4 / Mathlib formalization of the classical n -queens existence theorem: a placement of n mutually non-attacking queens on an $n \times n$ board exists if and only if $n \neq 2$ and $n \neq 3$ (Pauls 1874; explicit constructions by Hoffman–Loessi–Moore 1969 and Bernhardsson 1991). The theorem is *not new*; the contribution of this report is (i) the machine-checked artifact `ProofLab.NQueensTheorem.n_queens_exists_iff`, whose statement was frozen before any proof was attempted and whose axiom footprint is exactly `{propext, Classical.choice, Quot.sound}`; (ii) a 0-indexed transcription of Bernhardsson’s construction in which every case reduces to a linear-arithmetic goal closed by `omega`; (iii) an adversarial review with negative controls showing each clause of the formal definition is load-bearing; and (iv) an account of the gate-by-gate process (five agent roles under a human board) that produced it in under one hour of wall-clock time, including one transcription bug that survived a brute-force check over 997 boards and was caught only by the type checker. The lab makes **no external claim**. Residual risks are listed explicitly.

Status and claim posture

This is an internal report of Open Math Lab, produced under the lab’s claim policy (`docs/CLAIM_POLICY.md`): the default position is *no claim*, and only the human board may authorize external communication. Nothing here is a research result. Every mathematical statement below has one of two labels: **Lean-checked** (a named declaration in Appendix A, verified by the kernel) or **informal** (prose, evidence, or process narrative). The full process record, including dead ends and an erratum, is preserved in the repository and summarized in Section 7.

Contents

Status and claim posture	1
1 Introduction	2
2 Historical and formal context	3
3 The statement	4

*Paperclip agents (Claude): Problem Scout, Formalist, Attack Lead, Adversarial Reviewer, Research Director (editor of this report). Human board: Paul. Repository: <https://github.com/agentforce314/open-math-lab>.

4	The construction	5
4.1	Why three formulas rather than six residue classes	5
4.2	The even-board interface	5
4.3	E1: n even, $n \equiv 0, 4 \pmod{6}$	5
4.4	E2: n even, $n \equiv 2 \pmod{6}$	6
4.5	Odd boards: corner extension	7
4.6	Assembly	7
4.7	Dead ends (informal)	7
5	The formalization	8
5.1	Architecture	8
5.2	Verification	8
5.3	Two deviations from the attack log, and an erratum	9
6	Adversarial review	9
6.1	Statement fidelity	9
6.2	Negative controls	10
6.3	Machine-trust boundary	10
7	Process	10
7.1	Pipeline and gates	10
7.2	Timeline and budget	11
7.3	Artifacts	11
8	What is not claimed; residual risks	11
9	Lessons	12
10	Conclusion	12
A	Complete Lean source	13
B	Verification transcripts	17
B.1	Reviewer’s Lean checks	17
B.2	Attack Lead’s brute-force evidence (not proof)	18
C	Ticket timeline	18

1 Introduction

“The n -queens problem” names at least five different mathematical questions. Table 1 lists the five that the lab’s Problem Scout identified and scored on the lab’s feasibility rubric before any work was assigned. Exactly one of them, the *existence* question, is both settled in the literature and finitely encodable in a way that a proof assistant can check in a single sprint. That one was funded; the others are recorded as out of scope and remain untouched.

The board’s instruction was to “solve the n -queens problem end to end using the whole process, assuming the problem is new to you.” We read “new to you” as an instruction about *process*, not about pretending: the repository already contained a small Level A artifact for boards $n \leq 4$ (Section 2), which every ticket cites as prior art. What was genuinely new for the lab was the namesake theorem itself, which had been explicitly left unproved.

	Reading	Score	Disposition
(a)	Existence: a placement exists $\iff n \notin \{2, 3\}$	88	target ; known-classical, formalize-only
(b)	Counting $Q(n)$ (OEIS A000170; $Q(8) = 92$)	54	out of scope (no closed form known)
(c)	Asymptotics $Q(n) = ((1 \pm o(1)) ne^{-\alpha})^n$ (Simkin 2021)	28	refused
(d)	Completion is NP-complete (Gent–Jefferson–Nightingale 2017)	36	refused
(e)	Toroidal queens exist $\iff \gcd(n, 6) = 1$ (Pólya 1918)	80	different theorem; out of scope

Table 1: Disambiguation by the Problem Scout (issue OPE-3). Scores are on the lab rubric (five axes $\times$ 0–20). Only (a) was funded.

What this report is. A formalization report and a process report. Section 3 fixes the formal statement. Section 4 gives the mathematics of the construction in the 0-indexed form that was actually formalized, with each lemma tagged by its Lean name. Section 5 describes the Lean development and its verification. Section 6 reproduces the adversarial review, including the negative controls. Section 7 records the pipeline, gates, timeline, and budgets. Section 8 lists what is not claimed and what remains at risk. Section 9 draws process lessons. Appendix A is the complete Lean source; Appendices B–C are verification transcripts and the ticket timeline.

What this report is not. It is not a claim of novelty, priority, or difficulty. It does not address readings (b)–(e). It does not assert that the formal predicate `NonAttacking` is “the” definition of the problem; that identification is a human judgement, made independently three times (Scout, Director, Reviewer) and exercised with counterexamples, but not certified by the machine.

2 Historical and formal context

History. Bezzel posed the eight-queens puzzle in 1848 and Nauck reported the 92 solutions in 1850; the Gauss–Schumacher correspondence of that year concerns the 8×8 count only, not the general theorem. Pauls (1874) first proved existence for all $n \geq 4$. Ahrens (1910) gave a textbook treatment. Hoffman, Loessi and Moore (1969) and Bernhardsson (1991) gave explicit closed-form constructions split on $n \bmod 6$; the latter’s one-page note is the source transcribed here. Bell and Stevens (2009) survey the field and confirm that (a) is settled, (b) is open-ended, and (e) is Pólya’s theorem.

Mathlib. At the pinned Mathlib commit (a719ba5c31, Lean v4.10.0) a grep of Mathlib, Archive and Counterexamples for the pattern

```
nqueens | n_queens | eightqueens | non.?attacking | queens?
```

returns no mathematical hit (the only match is a “Queen Mary University” comment). A negative control (`isTuranMaximal_iff_nonempty_iso_turanGraph`) fires, so the grep is sound. This screen is local to the pin; it is not a survey of the wider Lean ecosystem.

Prior art in this repository. `ProofLab/NQueens.lean` (upstream PR #174) defines a *list* encoding `IsNQueens : List (Fin n) → Prop` (length n , Nodup, both diagonal families) and proves `queens_two_none`, `queens_three_none`, and the two $n = 4$ witnesses $[1, 3, 0, 2]$ and

[2, 0, 3, 1]. The namesake theorem was left as a residual there. The present work uses a *function* encoding in a new file and re-proves the $n = 2, 3$ cases on that encoding, so no bridge lemma between the two encodings is needed—and none was proved (Residual 1 in Section 8).

3 The statement

Throughout, boards are 0-indexed: rows and columns are $\{0, \dots, n-1\}$, realised in Lean as $\text{Fin } n$. A *placement* is a function $q : \text{Fin } n \rightarrow \text{Fin } n$; row i holds one queen, in column $q(i)$. One queen per row is thus built into the encoding.

Definition 3.1 (NonAttacking, Lean-checked definition). A placement q is *non-attacking* if

- (i) (columns) q is injective;
- (ii) (anti-diagonals) for all $i \neq j$, $i + q(i) \neq j + q(j)$;
- (iii) (main diagonals) for all $i \neq j$, $i + q(j) \neq j + q(i)$.

All sums are in $\mathbb{N}$ after coercion from $\text{Fin } n$; they are *not* taken modulo n .

Clause (iii) is the subtraction-free form of $i - q(i) \neq j - q(j)$: over $\mathbb{Z}$ the two are the same inequality rearranged, and since every term is a natural number no coercion subtlety arises. Writing it this way keeps the whole development inside *omega*’s fragment (linear arithmetic over $\mathbb{N}$ with literal coefficients).

Convention 3.2 ($n = 0$). $\text{Fin } 0 \rightarrow \text{Fin } 0$ has exactly one function and Definition 3.1 is vacuous on it. The empty board therefore *has* a placement. Consequently the theorem is stated with right-hand side $n \neq 2 \wedge n \neq 3$, which is total in n , rather than the textbook “ $n = 1$ or $n \geq 4$ ”. For $n \geq 1$ the two are the same theorem. This convention was fixed by the Scout, confirmed by the Director, and re-derived independently by the Reviewer.

Theorem 3.3 (*n_queens_exists_iff*, Lean-checked). For every $n \in \mathbb{N}$,

$$(\exists q : \text{Fin } n \rightarrow \text{Fin } n, \text{NonAttacking}(q)) \iff (n \neq 2 \wedge n \neq 3).$$

Theorem 3.4 (*queens_exists_of_four_le*, Lean-checked). For every $n \geq 4$ there is a non-attacking placement $q : \text{Fin } n \rightarrow \text{Fin } n$.

The Lean text of the definition and both theorem headers is reproduced verbatim below. It was *frozen* at the statement-pin gate (issue OPE-4) before any proof work began, and was diffed byte-for-byte against the frozen text at the proof gate and again at review.

```
def NonAttacking {n : ℕ} (q : Fin n → Fin n) : Prop :=
  Function.Injective q ∧
  ∀ i j : Fin n, i ≠ j →
    (i : ℕ) + (q i : ℕ) ≠ (j : ℕ) + (q j : ℕ) ∧
    (i : ℕ) + (q j : ℕ) ≠ (j : ℕ) + (q i : ℕ)

theorem queens_exists_of_four_le (n : ℕ) (hn : 4 ≤ n) :
  ∃ q : Fin n → Fin n, NonAttacking q

theorem n_queens_exists_iff (n : ℕ) :
  (∃ q : Fin n → Fin n, NonAttacking q) ↔ (n ≠ 2 ∧ n ≠ 3)
```

Theorem 3.4 was split out at the pin gate as an optional target because it isolates the only hard direction; the $n \leq 3$ cases are finite.

4 The construction

This section is the mathematics as it was formalized. It follows the Attack Lead’s log (issue OPE-5), with the two corrections recorded in its erratum (Section 5.3) already applied. Every lemma is tagged with the Lean declaration that checks it. Throughout, $h := \lfloor n/2 \rfloor$; for even n , $n = 2h$.

4.1 Why three formulas rather than six residue classes

Bernhardsson’s construction is usually presented as a case split on $n \bmod 6$. The Attack Lead chose a coarser partition: two closed formulas for even boards and one generic *corner extension* that handles all odd boards uniformly. The reason is tactical rather than mathematical: every odd class then shares a single lemma whose only hypothesis is that the even board has no queen on its main diagonal, and the whole proof stays in one tactic flavour (`split_ifs <;> omega`). No modular arithmetic library (`ZMod`, `Nat.ModEq`) is needed anywhere. The alternative “knight” formula $r \mapsto 2r \bmod n$ for $\gcd(n, 6) = 1$ was machine-checked but deliberately not formalized, because it would have required a second proof technique without removing any case from the plan.

4.2 The even-board interface

Definition 4.1 (`GoodEven`, Lean-checked definition). For $n \in \mathbb{N}$ and $f : \mathbb{N} \rightarrow \mathbb{N}$, `GoodEven( $n, f$ )` holds if, for all $r, s < n$:

- (a) $f(r) < n$;
- (b) $f(r) = f(s) \Rightarrow r = s$;
- (c) $r \neq s \Rightarrow r + f(r) \neq s + f(s)$;
- (d) $r \neq s \Rightarrow r + f(s) \neq s + f(r)$;
- (e) $f(r) \neq r$ (no fixed point: no queen on the main diagonal through the origin).

Clauses (a)–(d) say that f restricted to $\{0, \dots, n-1\}$ is a non-attacking placement in the sense of Definition 3.1; clause (e) is the side condition the corner extension needs.

Lemma 4.2 (`nonAttacking_of_nat`, Lean-checked; the only `Fin` lemma). *If $f : \mathbb{N} \rightarrow \mathbb{N}$ satisfies (a)–(d) on $\{0, \dots, n-1\}$ then $i \mapsto \langle f(i), (a) \rangle$ is a `NonAttacking placement` $\text{Fin } n \rightarrow \text{Fin } n$.*

Everything else in the development is a statement about $\mathbb{N} \rightarrow \mathbb{N}$; Lemma 4.2 is the single crossing into dependent types.

4.3 E1: n even, $n \equiv 0, 4 \pmod{6}$

Definition 4.3 (`f1`). $f_1(r) = 2r + 1$ if $r < h$, and $f_1(r) = 2r - n$ otherwise.

Odd columns $1, 3, \dots, n-1$ fill the top half, even columns $0, 2, \dots, n-2$ the bottom half. For $n = 4$ this is $[1, 3, 0, 2]$, the Level A witness.

Lemma 4.4 (`e1_lt`, `e1_inj`, `e1_main`, `e1_no_fix`; Lean-checked). *For even n , f_1 satisfies (a), (b), (d), (e).*

Informal argument. (a) $r < h \Rightarrow 2r + 1 \leq n - 1$; $r \geq h \Rightarrow 2r - n \leq n - 2$. (b) Within a half, $2r + 1 = 2s + 1$ or $2r - n = 2s - n$ forces $r = s$; across halves $2r + 1 = 2s - n$ equates an odd and an even number. (d) In $\mathbb{Z}$, $r - f_1(r)$ is $-r - 1$ on the top half and $n - r$ on the bottom; within a half this is injective, and across halves $-r - 1 = n - s$ gives $s - r = n + 1 > s$, impossible. (e) $2r + 1 = r$ is impossible in $\mathbb{N}$; $2r - n = r$ gives $r = n$, contradicting $r < n$. $\square$

Lemma 4.5 (e1_anti, Lean-checked). *If $n \bmod 6 \in \{0, 4\}$ then f_1 satisfies (c).*

Informal argument. Within a half, $3r + 1 = 3s + 1$ or $3r - n = 3s - n$ forces $r = s$. Across halves ($r < h \leq s$), $3r + 1 = 3s - n$ gives $3(s - r) = n + 1$, i.e. $n \equiv 2 \pmod{3}$, excluded by hypothesis. $\square$

Lemma 4.5 is the only place the residue class is used, and the restriction is sharp: for $n = 8$, rows 1 and 4 of f_1 share anti-diagonal 4.

Proposition 4.6 (goodEven_f1, Lean-checked). *If $n \bmod 6 \in \{0, 4\}$ then $\text{GoodEven}(n, f_1)$.*

4.4 E2: n even, $n \equiv 2 \pmod{6}$

Here $h \bmod 3 = 1$, and for $n \geq 8$, $h \geq 4$. Bernhardsson's second formula contains a " $x \bmod n$ " wrap; the Attack Lead resolved the wrap into explicit branches, since $x \bmod n$ with variable n lies outside omega's fragment.

Definition 4.7 (f_2).

$$f_2(r) = \begin{cases} 2r + h - 1 & r < h, 2r \leq h \quad (\text{P1}) \\ 2r - h - 1 & r < h, 2r > h \quad (\text{P2}) \\ 2r + 2 - h & r \geq h, 2r < 3h - 2 \quad (\text{P3}) \\ 2r + 2 - 3h & r \geq h, 2r \geq 3h - 2 \quad (\text{P4}) \end{cases}$$

For $n = 8$: $[3, 5, 7, 1, 6, 0, 2, 4]$; for $n = 14$: $[6, 8, 10, 12, 0, 2, 4, 9, 11, 13, 1, 3, 5, 7]$. The second half is the 180° rotation of the first, $f_2(n - 1 - r) = n - 1 - f_2(r)$, which is how the formula was derived but is not used by the proof. Table 2 lists the linear forms that omega sees in each piece.

piece	$f_2(r)$	$r + f_2(r)$ (anti)	$r - f_2(r)$ (main, in $\mathbb{Z}$)	parity of $f_2(r)$
P1	$2r + h - 1$	$3r + h - 1$	$-r - h + 1$	$h - 1$
P2	$2r - h - 1$	$3r - h - 1$	$-r + h + 1$	$h - 1$
P3	$2r - h + 2$	$3r - h + 2$	$-r + h - 2$	h
P4	$2r - 3h + 2$	$3r - 3h + 2$	$-r + 3h - 2$	h

Table 2: Piecewise-linear data for f_2 . Values are as integers; the Lean definition orders the operations so that no piece truncates in $\mathbb{N}$ on its own range (Section 5.3).

Lemma 4.8 (e2_lt, e2_inj, e2_anti; Lean-checked). *If $n \bmod 6 = 2$ then f_2 satisfies (a), (b), (c).*

Informal argument. (a) Piece bounds: $P1 \leq 2h - 1 = n - 1$; $P2 \leq h - 3$; $P3 < n$; $P4 \leq h$. (b) Within a piece $2r + c = 2s + c$; P1/P2 and P3/P4 cross-cases give $s - r = h$ against the piece ranges; top-vs-bottom cross-cases have opposite parity (last column of Table 2). (c) Within a piece $3r + c = 3s + c$. Cross-piece constant differences are $2h, 2h - 3, 4h - 3, 3, 2h - 3, 2h$; each $3(s - r) = \pm \text{const}$ forces $3 \mid 2h$ or $3 \mid 4h$, impossible as $h \equiv 1 \pmod{3}$, except P2-P3 where $3(s - r) = -3$ gives $s = r - 1 < h$, contradicting $s \geq h$. $\square$

Lemma 4.9 (e2_main, Lean-checked; needs $8 \leq n$). *If $n \bmod 6 = 2$ and $n \geq 8$ then f_2 satisfies (d).*

Informal argument. Within a piece, $-r + c = -s + c$. P1P2 and P3P4: $s - r = 2h \geq n$. P1P4: $s - r = 4h - 3 \geq n$. P2P3: $r - s = 3$ with $r < h \leq s$. P1P3: $s - r = 2h - 3$ with $s < (3h - 2)/2$ forces $h < 4$. P2P4: $s - r = 2h - 3$ with $r > h/2, s \leq 2h - 1$ forces $h \leq 3$. Hence $h \geq 4$ suffices. $\square$

Lemma 4.10 (e2_no_fix, Lean-checked; needs $4 \leq n$). If $n \bmod 6 = 2$ and $n \geq 4$ then f_2 satisfies (e).

Informal argument. A fixed point would be P1 $r = 1 - h$; P2 $r = h + 1 > h$; P3 $r = h - 2 < h$; P4 $r = 3h - 2 \geq 2h$ (using $h \geq 2$). Each contradicts its piece's range. $\square$

The hypothesis $4 \leq n$ in Lemma 4.10 is genuinely needed: at $n = 2$, $f_2(0) = 0$ and $f_2(1) = 1$. It was missing from the attack log's typed lemma (Section 5.3); in the assembly it is discharged from $8 \leq n$.

Proposition 4.11 (goodEven_f2, goodEven_exists; Lean-checked). If $n \bmod 6 = 2$ and $n \geq 8$ then $\text{GoodEven}(n, f_2)$. Consequently every even $n \geq 4$ admits some f with $\text{GoodEven}(n, f)$.

(For even $n \geq 4$, $n \bmod 6 \in \{0, 2, 4\}$; the class 2 case has $n \geq 8$ automatically.)

4.5 Odd boards: corner extension

Definition 4.12 (corner). For $m \in \mathbb{N}$ and $g : \mathbb{N} \rightarrow \mathbb{N}$, $\text{corner}(m, g)(r) = g(r)$ if $r < m$, and $= m$ otherwise.

Board $m + 1$ is board m with one more queen in the corner (m, m) .

Lemma 4.13 (corner_lt, corner_inj, corner_anti, corner_main; all Lean-checked). If $\text{GoodEven}(m, g)$ then $\text{corner}(m, g)$ satisfies (a)–(d) on the set $\{0, \dots, m\}$.

Informal argument. Both rows below m : inherited from g . One row equal to m : (a) $m < m + 1$; (b) $g(r) = m$ contradicts $g(r) < m$; (c) $r + g(r) \leq 2m - 2 < 2m = m + m$; (d) $r + m \neq m + g(r) \iff g(r) \neq r$, which is exactly clause (e) of GoodEven . $\square$

The last line is the whole point of clause (e): without it the corner queen would lie on the main diagonal through some g -queen. This is also why a toroidal (modular) solution cannot be corner-extended (Section 4.7).

4.6 Assembly

Proof of Theorem 3.4 (queens_exists_of_four_le). Write $n = 2k$ or $n = 2k + 1$. If even, take the f from goodEven_exists and apply Lemma 4.2. If odd, $2k \geq 4$; take g with $\text{GoodEven}(2k, g)$, form $\text{corner}(2k, g)$, apply Lemma 4.13 and then Lemma 4.2. $\square$

Proof of Theorem 3.3 (n_queens_exists_iff). $(\Rightarrow)$ $\text{Fin } 2 \rightarrow \text{Fin } 2$ has 4 functions and $\text{Fin } 3 \rightarrow \text{Fin } 3$ has 27; after unfolding NonAttacking , kernel decide refutes each (queens_two_none', queens_three_none'). $(\Leftarrow)$ If $n < 4$ then $n \in \{0, 1\}$ and the identity is a placement (by decide); otherwise Theorem 3.4. $\square$

4.7 Dead ends (informal)

The attack log records three routes that were tried and rejected, with the obstruction in each case:

- 1-indexed “list surgery” for $n \bmod 6 \in \{2, 3\}$ (“swap $1 \leftrightarrow 3$, move 5 to the end”). Transcribed to 0-indexed form it is a five-piece function with three single-point pieces, and the $n \bmod 6 = 3$ list is *not* a corner extension of the $n \bmod 6 = 2$ list (for $n = 9$ the last row holds column 2, not 8). Rejected for Bernhardsson's closed formula, whose companion is the corner extension. Obstruction: more cases and no reuse, not a mathematical failure.

2. *Corner-extending a toroidal solution.* Any modular solution $r \mapsto (ar + b) \bmod m$ has a queen on every wrapped diagonal, in particular on the actual main diagonal $r = q(r)$, so clause (e) fails and the corner queen is attacked. Exact obstruction.
3. *A single mod n formula for even n .* $(2r+c) \bmod n$ is never injective for even n ($\gcd(2, n) = 2$), so no one-line modular argument exists for even boards; the $r < h$ split is unavoidable.

5 The formalization

5.1 Architecture

The development is one file, `proofs/lean-project/ProofLab/NQueensTheorem.lean` (272 lines, Appendix A), in namespace `ProofLab.NQueensTheorem`. It contains five definitions (`NonAttacking`, `GoodEven`, f_1 , f_2 , `corner`) and twenty-two theorems. Its shape follows the attack plan exactly:

- one transfer lemma from $\mathbb{N} \rightarrow \mathbb{N}$ into $\text{Fin } n \rightarrow \text{Fin } n$ (Lemma 4.2);
- an interface record `GoodEven` so that `E1`, `E2` and the corner step compose without knowing each other's formulas;
- every arithmetic lemma proved by the same three-token script `simp only [f]; split_ifs <;> omega`, one line each;
- the two finite cases by kernel decide on the function space, after unfold `NonAttacking` (plain decide does not see through the def);
- three examples as sanity checks: the Level A witness satisfies the predicate, $f_1(4) = [1, 3, 0, 2]$, $f_2(8) = [3, 5, 7, 1, 6, 0, 2, 4]$.

The `unusedVariables` linter is disabled for the arithmetic section because `omega` consumes the bound hypotheses $r < n, s < n$ from context without a syntactic reference; the Formalist checked with `clear` that they are load-bearing (e.g. `e1_main` fails at $n = 4, r = 0, s = 5$ without $s < n$).

5.2 Verification

The proof was built, from a deleted module cache, by three parties on the same machine (Lean 4.10.0, arm64 macOS; Mathlib a719ba5c31): the Director at the proof gate, the Reviewer during review, and the Director again on the integrated close-out branch. Each time:

✓ [2240/2240] Built `ProofLab.NQueensTheorem`
Build completed successfully.

with no sorry warning, and

```
'ProofLab.NQueensTheorem.n_queens_exists_iff' depends on axioms:
  [propext, Classical.choice, Quot.sound]
'ProofLab.NQueensTheorem.queens_exists_of_four_le' depends on axioms:
  [propext, Quot.sound]
```

A `grep` of the file for the pattern

```
sorry|admit|axiom|native_decide|implemented_by|extern|unsafe|partial
```

returns only the expected `decide` calls. `Classical.choice` enters through Mathlib's `Fintype` decidability instances used by `decide` on the finite function spaces; `queens_exists_of_four_le` does not need it.

5.3 Two deviations from the attack log, and an erratum

The Formalist reported two places where the landed Lean differs from the Attack Lead’s *typed* lemma list; the Reviewer confirmed both independently.

(a) **$\mathbb{N}$ -truncation in the typed f_2 .** The log typed pieces P3/P4 as $2 * r - n / 2 + 2$ and $2 * r - 3 * (n / 2) + 2$. In $\mathbb{N}$ these parse as $(2r - 3h) + 2$ and truncate whenever $2r \in \{3h - 2, 3h - 1\}$: for $n = 8, r = 5$ the typed form gives 2 instead of 0; for $n = 14, r = 10$ it gives 2 instead of 1. The Reviewer’s own script found 166 mismatched rows over $8 \leq n \leq 1000, n \equiv 2 \pmod{6}$ —exactly one row per board—and the typed form would have broken injectivity (at $n = 8$, rows 5 and 6 would collide). The Attack Lead’s Python check used signed integers and so never exercised the $\mathbb{N}$ form. Lean’s f_2 is written $2 * r + 2 - n / 2$ and $2 * r + 2 - 3 * (n / 2)$, which agree with the signed formula on every branch where they are evaluated. The mathematics in the log and the brute-force evidence were correct; only the $\mathbb{N}$ transcription was wrong. The log now carries an erratum header and the corrected forms.

(b) **A missing hypothesis.** The typed `e2_no_fix` had no size hypothesis and is false at $n = 2$; the log’s justification “(as $h \geq 2$)” was unsupported. Lean adds $4 \leq n$, discharged by `omega` from the assembly’s $8 \leq n$. The Reviewer tabulated every landed declaration against the log (Table 3): this is the only hypothesis delta in the file, and nothing was weakened.

declaration(s)	hypotheses in log	delta in Lean
<code>nonAttacking_of_nat</code>	<code>hlt hinc hanti hmain</code>	none
<code>e1_lt, e1_inj, e1_main, e1_no_fix</code>	$n \bmod 2 = 0$	none
<code>e1_anti, goodEven_f1</code>	$n \bmod 6 \in \{0, 4\}$	none
<code>e2_lt, e2_inj, e2_anti</code>	$n \bmod 6 = 2$	none
<code>e2_main, goodEven_f2</code>	$n \bmod 6 = 2, 8 \leq n$	none
<code>e2_no_fix</code>	$n \bmod 6 = 2$	$+ 4 \leq n$ (needed)
<code>goodEven_exists</code>	$4 \leq n, n \bmod 2 = 0$	none
<code>corner_*</code>	sketched	made concrete; none
<code>queens_exists_of_four_le</code>	$4 \leq n$ (frozen)	none

Table 3: Hypothesis diff, attack log versus landed Lean (from the adversarial review).

6 Adversarial review

The lab’s rule is that no mathematical claim is closed without *both* a green Lean build *and* written approval from an Adversarial Reviewer with veto power and a non-empty residual-risk list. The Reviewer (issue OPE-7) was given the artifact pinned to a commit and a binding checklist; the review file and its scratch Lean file are in the repository (`attacks/n-queens-20260912-180329/ADVERSARIAL_REVIEW.md`, `review_checks.lean`). Verdict: **approve with residuals**.

6.1 Statement fidelity

The Reviewer read Definition 3.1 cold and matched each clause to the chess rule (rows = domain, columns = codomain, injectivity = distinct columns, $i + q(i)$ = anti-diagonal index, $i + q(j) = j + q(i)$ = shared main diagonal, sums in $\mathbb{N}$ not $\text{Fin } n$). Two properties were then established by kernel `decide`:

- *Not vacuous*: the predicate is unsatisfiable at $n = 2, 3$.

- *Not over-strong*: concrete boards for $n = 4, 5, 6, 8, 14$ are accepted, including $[1, 3, 0, 2]$, which would fail twice if the sums were taken modulo 4.

6.2 Negative controls

To show that each clause is load-bearing, the Reviewer enumerated all 256 functions $\text{Fin } 4 \rightarrow \text{Fin } 4$ (informally, in Python) to select boards that violate *exactly one* clause, then checked in Lean that NonAttacking rejects each, and that the predicate with that one clause removed accepts it (Table 4). The enumeration also confirms that exactly two of the 256 functions are accepted, the two classical $n = 4$ solutions.

board	violates only	NonAttacking	weakened predicate accepts
$[0, 1, 2, 3]$	main diagonal	rejected	NoMain
$[3, 2, 1, 0]$	anti-diagonal	rejected	—
$[1, 3, 2, 0]$	anti-diagonal	rejected	NoAnti
$[0, 0, 0, 0]$	injectivity	rejected	NoInj
$[1, 1, 0, 2]$	injectivity <i>and</i> anti-diagonal	rejected	(not single-clause; replaced)

Table 4: Negative controls, all by kernel decide (`review_checks.lean`). Any control accepted by NonAttacking would have been an automatic reject.

6.3 Machine-trust boundary

“Lean checked it” means: the Lean 4.10.0 kernel accepted a term of type

$$\forall n, (\exists q, \text{NonAttacking } q) \leftrightarrow (n \neq 2 \wedge n \neq 3)$$

from the three standard axioms plus the pinned Mathlib’s definitions. This certifies that the formal proposition follows from those axioms, modulo trusting the kernel, the `elan`-installed toolchain binary, the Mathlib `.olean` cache on one machine, and the lake build graph. It does *not* certify that NonAttacking is the right predicate—that is the human judgement documented above—nor anything about readings (b)–(e), nor any link to the Level A list encoding. No `native_decide` or `ofReduceBool` shortcut was used anywhere.

7 Process

7.1 Pipeline and gates

The lab runs as five Paperclip agent roles under a human board. For a board-named problem the Scout shortlist gate is satisfied by the board, but every other gate still fires. Each specialist was woken *one at a time*; the Director re-verified the artifact at each gate before releasing the next ticket, and the Director’s own checks were logged on the parent issue so the Reviewer could see what was and was not yet independent.

1. **Board** (OPE-2): names the problem; asks for the whole process; approves user-local tooling.
2. **Director**: installs Lean 4.10.0 via `elan` (one macOS-26 linker workaround, documented); frames the readings; opens the child chain; sets kill criteria.
3. **Problem Scout** (OPE-3): disambiguation, rubric score, literature handles, Mathlib screen, Level B statement pin, top three definition risks, budget. Does not solve.
4. **Formalist, statement pin** (OPE-4): the frozen text compiles with a single `sorry; #print axioms` shows `sorryAx`, as it must for a stub.

5. **Attack Lead** (OPE-5): 0-indexed construction, informal proofs shaped for omega, brute-force evidence to $n = 1000$, Lean-shaped lemma list, dead-end map. Labelled *status: informal*.
6. **Formalist, proof** (OPE-6): closes both theorems in one wave (cap: two), reports the two deviations.
7. **Adversarial Reviewer** (OPE-7): fresh rebuild, axioms, fidelity, negative controls, hypothesis diff, scope sweep, residuals. Veto power.
8. **Director close-out** (OPE-2): integrates branches, records the erratum, moves the ledger and catalog to *formalized*, prepares the claim packet (`mathforge claim prepare`) with recommendation *no claim*.

7.2 Timeline and budget

All work took place on 12 September 2026. From the board’s comment (17:37 UTC) to the close-out disposition (18:34 UTC) is 57 minutes of wall-clock time across eight heartbeats; the per-ticket commit times are in Appendix C. Token budgets were set per ticket (Scout and stub small; Attack Lead cap 60k, reported $\approx 45k$ used; Formalist proof wave 1 $\approx 120k$ of a two-wave cap; Reviewer $\approx 80k$); the Scout’s overall estimate of $\approx 300k$ was accurate to within a few percent. The Director’s kill criterion (stop if more than one of the four proof blocks remained open after wave 2) was never triggered.

Two facts keep this timeline in proportion. The theorem is classical and its construction is a one-page note in the literature; and the repository already had the Lean toolchain configuration, a Mathlib cache, a claim policy, a review checklist, and a role definition for every participant. The hour measures the *process*, not the mathematics.

7.3 Artifacts

All artifacts are on the fork `agentforce314/open-math-lab`, branch `ope/2-nqueens-closeout` (pull request #8, which integrates #3, #5, #6, #7):

- `catalog/problems/n-queens/STATEMENT.md`, `Dossier.json` — Scout dossier and pin;
- `attacks/n-queens-20260912-180329/` — attack log (with erratum), strategies, `verify_construction.py` and its output, the adversarial review, `review_checks.lean` and its output;
- `proofs/lean-project/ProofLab/NQueensTheorem.lean` — the proof;
- `problems/n-queens/CLAIM_PACKET.md` — claim packet (no claim);
- `docs/PROBLEM_LEDGER.md` — lifecycle record;
- `proofs/n-queens/paper/` — this report and its build script.

8 What is not claimed; residual risks

Not claimed. New mathematics; priority in Mathlib or elsewhere; anything about readings (b)–(e); that `NonAttacking` is the unique correct formalization; equivalence with the Level A list encoding; any “AI solved n -queens” framing.

Residual risks (from the review, board-facing).

1. No Lean bridge `IsNQueens (List.ofFn q)  $\leftrightarrow$  NonAttacking q`; two encodings coexist.
2. Only reading (a) is proved.
3. Definition fidelity is a human judgement, made three times; a further reader is cheap.
4. Toolchain trust is single-machine; no `lean4checker` re-check of the `.olean`.
5. No priority or novelty; the Mathlib screen is local to the pin.

6. The attack log’s erratum (Section 5.3), now fixed, is kept on record.
7. $n \neq 2 \wedge n \neq 3$ includes the vacuous $n = 0$ (Convention 3.2); do not paraphrase as “ $n = 1 \vee n \geq 4$ ” without the caveat.
8. An unused `import ProofLab.NQueens` in the new file; harmless, since the axiom print rather than the build is the gate.

9 Lessons

1. **Brute-force evidence must run on the typed formula.** The Attack Lead’s check evaluated the pin’s three clauses literally over all ordered pairs on 997 boards and passed on the first run—and still missed a bug, because it computed with signed integers while the lemma list was typed in $\mathbb{N}$. Future attack tickets should evaluate the exact $\mathbb{N}$ -typed expression (or generate the Python from the Lean text).
2. **Freeze the statement before the proof.** Diffing the frozen text at every gate cost seconds and removed an entire class of “proved a different theorem” failures.
3. **Isolate the hard direction.** Splitting off `queens_exists_of_four_le` at the pin gate let the finite cases, the construction, and the review proceed as separate units of work.
4. **One interface, one tactic.** The GoodEven record and the `split_ifs <=> omega` discipline meant that the Formalist’s wave was almost entirely transcription, and that the Reviewer could diff hypotheses mechanically.
5. **The review gate is not a formality.** Both deviations were real defects in a document that had passed a machine check; they were caught by the type checker and independently confirmed by the Reviewer, and the ticket that was funded to prevent this class of error (the transcription step) was the one that introduced it.
6. **Kernel decide is enough for small finite cases** (4 and 27 functions) and keeps the axiom footprint standard; `native_decide` was never needed and is not an accepted gate.

10 Conclusion

The lab set out to run its full process on a problem chosen by the board, and did so: a scouted and disambiguated target, a frozen statement, an informal construction with machine evidence and an honest dead-end map, a zero-sorry Lean proof with a standard axiom footprint, an adversarial review with negative controls, and a close-out with a claim packet recommending no claim. The theorem was known in 1874. What the lab can now say, and could not before, is that the specific formal proposition in Section 3 follows from Lean’s three standard axioms, and that the definition it is about has been read and stress-tested by three independent readers.

References

- [1] W. Ahrens, *Mathematische Unterhaltungen und Spiele*, 2nd ed., Teubner, 1910.
- [2] J. Bell and B. Stevens, “A survey of known results and research areas for n -queens,” *Discrete Mathematics* 309(1):1–31, 2009.
- [3] B. Bernhardsson, “Explicit solutions to the N -queens problem for all N ,” *ACM SIGART Bulletin* 2(2):7, 1991.
- [4] I. P. Gent, C. Jefferson, and P. Nightingale, “Complexity of n -queens completion,” *Journal of Artificial Intelligence Research* 59:815–848, 2017.

- [5] E. J. Hoffman, J. C. Loessi, and R. C. Moore, “Constructions for the solution of the m queens problem,” *Mathematics Magazine* 42(2):66–72, 1969.
- [6] E. Pauls, “Das Maximalproblem der Damen auf dem Schachbrette,” *Deutsche Schachzeitung* 29:129–134, 257–267, 1874.
- [7] M. Simkin, “The number of n -queens configurations,” arXiv:2107.13460, 2021.
- [8] The Mathlib Community, *Mathlib*, commit a719ba5c31 (Lean v4.10.0), 2024.
- [9] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” CADE 28, 2021.

A Complete Lean source

proofs/lean-project/ProofLab/NQueensTheorem.lean, as integrated on the close-out branch (identical to the reviewed commit 8ae158a). Included verbatim from the repository at build time.

```

1 /-
2 n-queens – Level B namesake theorem (OPE-4 statement pin; OPE-6 proof, Formalist).
3
4 status: known-classical, formalize-only, **no novelty claim**.
5 Parent OPE-2 (board-named "solve the n-queens problem"); Scout dossier
6 OPE-3 (`catalog/problems/n-queens/STATEMENT.md`, "Exact Lean-shaped
7 statement (Level B pin)"); Attack Lead construction OPE-5
8 (`attacks/n-queens-20260912-180329/LOG.md`); Director gates released
9 2026-09-12.
10
11 `def NonAttacking`, `theorem queens_exists_of_four_le` and
12 `theorem n_queens_exists_iff` are **frozen** (byte-identical to the
13 OPE-4 pin). Everything else in this file is helper material owned by
14 OPE-6.
15
16 Why a new file rather than editing `ProofLab/NQueens.lean`: the Level A
17 witnesses there (PR #174; `IsNQueens` list encoding, `queens_two_none`,
18 `queens_three_none`, `queens_four`) are landed, fully proved, and cited
19 as prior art. Keeping them untouched keeps that module green and lets
20 `lake build ProofLab.NQueensTheorem` be the single gate for the
21 namesake. The `n = 2, 3` nonexistence here is re-proved on the function
22 encoding by kernel `decide` (4 resp. 27 functions), so no bridge lemma
23 to `IsNQueens` is needed.
24
25 Route (Attack Lead, OPE-5, 0-indexed throughout): the whole proof lives
26 in ` $\mathbb{N} \rightarrow \mathbb{N}$ ` and crosses into ` $\text{Fin } n$ ` exactly once (`nonAttacking_of_nat`).
27 Even boards use Bernhardsson's two closed formulas – `f1` for
28 ` $n \% 6 \in \{0, 4\}$ `, `f2` for ` $n \% 6 = 2$ ` (needs ` $8 \leq n$ ) – bundled as
29 `GoodEven` (bound, injective, anti-diagonal, main-diagonal, and the
30 load-bearing side condition "no queen on the main diagonal"). Odd boards
31 ` $n \geq 5$ ` are the even board ` $n - 1$ ` plus a queen in the corner
32 `(n-1, n-1)` (`corner`); the no-fixed-point condition is exactly what
33 keeps the corner queen off every main diagonal.
34
35 One deliberate deviation from the LOG's *typed* `f2`: its last two
36 pieces were written ` $2 * r - n / 2 + 2$ ` and ` $2 * r - 3 * (n / 2) + 2$ `,
37 which in ` $\mathbb{N}$ ` parse as ` $(2r - 3h) + 2$ ` and truncate when ` $2r \in \{3h-2,$ 
38  $3h-1\}$ ` (e.g. ` $n = 14, r = 10$ ` would give `2`, not `1`). The Python
39 verifier used signed ints and never saw this. Here they are written
40 ` $2 * r + 2 - n / 2$ ` and ` $2 * r + 2 - 3 * (n / 2)$ `, which agree with the

```

```

41 signed formula on every branch where they are evaluated.
42
43 Convention (`n = 0`): `NonAttacking` on `Fin 0 → Fin 0` is vacuously
44 true (the empty placement), so the right-hand side is `n ≠ 2 ∧ n ≠ 3`
45 and the statement is total in `n`. This is intentional per the Scout
46 pin, not an oversight.
47
48 `#print axioms n_queens_exists_iff` (verified 2026-09-12, Lean 4.10.0):
49 `[propext, Classical.choice, Quot.sound]` – no stub axiom. (The header
50 avoids the literal stub keyword so the OPE-6 grep gate on this file is empty.)
51 -/
52 import Mathlib.Data.Fin.Basic
53 import Mathlib.Logic.Function.Defs
54 import Mathlib.Tactic
55 import ProofLab.NQueens
56
57 namespace ProofLab.NQueensTheorem
58
59 /-! ## Definition (verbatim from the Scout pin) -/
60
61 /- `q i` is the column of the queen in row `i`. Non-attacking: distinct columns
62 (`Function.Injective q`, Mathlib `Logic/Function/Defs.lean` L101) and no two queens
63 on a common anti-diagonal (`i + q i`) or main diagonal (`i - q i`, written
64 subtraction-free as `i + q j ≠ j + q i`). Both diagonal families are load-bearing. -/
65 def NonAttacking {n : ℕ} (q : Fin n → Fin n) : Prop :=
66   Function.Injective q ∧
67   ∀ i j : Fin n, i ≠ j →
68     (i : ℕ) + (q i : ℕ) ≠ (j : ℕ) + (q j : ℕ) ∧
69     (i : ℕ) + (q j : ℕ) ≠ (j : ℕ) + (q i : ℕ)
70
71 /-! ## Lemma 0: the only `Fin` lemma. Transfer from `ℕ → ℕ`. -/
72
73 /- A `ℕ`-function with the three properties (bounded, injective, no shared
74 anti-diagonal or main diagonal) on `{0, ..., n-1}` gives a `NonAttacking` placement. -/
75 theorem nonAttacking_of_nat {n : ℕ} (f : ℕ → ℕ) (hlt : ∀ r, r < n → f r < n)
76   (hinj : ∀ r s, r < n → s < n → f r = f s → r = s)
77   (hanti : ∀ r s, r < n → s < n → r ≠ s → r + f r ≠ s + f s)
78   (hmain : ∀ r s, r < n → s < n → r ≠ s → r + f s ≠ s + f r) :
79   NonAttacking (fun i : Fin n => ⟨f i, hlt i.i.isLt⟩) := by
80   refine ⟨fun i j hij => ?_, fun i j hij => ?_⟩
81   · exact Fin.ext (hinj _ _ i.isLt j.isLt (by simp using congrArg Fin.val hij))
82   · have hij' : (i : ℕ) ≠ j := fun h => hij (Fin.ext h)
83     exact ⟨hanti _ _ i.isLt j.isLt hij', hmain _ _ i.isLt j.isLt hij'⟩
84
85 /-! ## Even boards: the `GoodEven` interface -/
86
87 /- The five even-board facts: bound, injective, anti-diagonal, main-diagonal,
88 and no queen on the main diagonal `r = f r` (needed by the corner step). -/
89 def GoodEven (n : ℕ) (f : ℕ → ℕ) : Prop :=
90   (∀ r, r < n → f r < n) ∧ (∀ r s, r < n → s < n → f r = f s → r = s) ∧
91   (∀ r s, r < n → s < n → r ≠ s → r + f r ≠ s + f s) ∧
92   (∀ r s, r < n → s < n → r ≠ s → r + f s ≠ s + f r) ∧ (∀ r, r < n → f r ≠ r)
93
94 /- E1 (`n` even, `n % 6 ∈ {0, 4}`): odd columns `1, 3, ...` in the top half,
95 even columns `0, 2, ...` in the bottom half. `n = 4` gives `[1, 3, 0, 2]`. -/
96 def f₁ (n r : ℕ) : ℕ := if r < n / 2 then 2 * r + 1 else 2 * r - n
97
98 /- E2 (`n` even, `n % 6 = 2`): Bernhardsson's second formula with the
99 `% n` wrap resolved into explicit branches. `n = 8` gives `[3,5,7,1,6,0,2,4]`. -/
100 def f₂ (n r : ℕ) : ℕ :=
101   if r < n / 2 then
102     (if 2 * r ≤ n / 2 then 2 * r + n / 2 - 1 else 2 * r - n / 2 - 1)

```

```

103 else
104   (if 2 * r < 3 * (n / 2) - 2 then 2 * r + 2 - n / 2 else 2 * r + 2 - 3 * (n / 2))
105
106 /-! ### E1 lemmas
107
108 Each is `simp only [f]; split_ifs <=> omega`. The `unusedVariables` linter is
109 disabled for this section: `omega` consumes `hr : r < n`, `hs : s < n` from the
110 context without a syntactic reference, and they *are* load-bearing (e.g.
111 `el_main` fails at `n = 4, r = 0, s = 5` without `hs`; checked with `clear`). -/
112
113 section EvenLemmas
114 set_option linter.unusedVariables false
115
116 theorem e1_lt (n : ℕ) (hn : n % 2 = 0) : ∀ r, r < n → f₁ n r < n := by
117   intro r hr; simp only [f₁]; split_ifs <=> omega
118
119 theorem e1_inj (n : ℕ) (hn : n % 2 = 0) :
120   ∀ r s, r < n → s < n → f₁ n r = f₁ n s → r = s := by
121   intro r s hr hs h; simp only [f₁] at h; split_ifs at h <=> omega
122
123 /- The one place the class restriction is used: for `n % 6 = 2` the cross-half
124 case `3(s - r) = n + 1` is solvable (`n = 8`, rows `1` and `4`). -/
125 theorem e1_anti (n : ℕ) (hn : n % 6 = 0 ∨ n % 6 = 4) :
126   ∀ r s, r < n → s < n → r ≠ s → r + f₁ n r ≠ s + f₁ n s := by
127   intro r s hr hs hrs; simp only [f₁]; split_ifs <=> omega
128
129 theorem e1_main (n : ℕ) (hn : n % 2 = 0) :
130   ∀ r s, r < n → s < n → r ≠ s → r + f₁ n s ≠ s + f₁ n r := by
131   intro r s hr hs hrs; simp only [f₁]; split_ifs <=> omega
132
133 theorem e1_no_fix (n : ℕ) (hn : n % 2 = 0) : ∀ r, r < n → f₁ n r ≠ r := by
134   intro r hr; simp only [f₁]; split_ifs <=> omega
135
136 theorem goodEven_f1 (n : ℕ) (hn : n % 6 = 0 ∨ n % 6 = 4) : GoodEven n (f₁ n) :=
137   have h2 : n % 2 = 0 := by omega
138   (e1_lt n h2, e1_inj n hn, e1_anti n hn, e1_main n h2, e1_no_fix n h2)
139
140 /-! ### E2 lemmas -/
141
142 theorem e2_lt (n : ℕ) (hn : n % 6 = 2) : ∀ r, r < n → f₂ n r < n := by
143   intro r hr; simp only [f₂]; split_ifs <=> omega
144
145 theorem e2_inj (n : ℕ) (hn : n % 6 = 2) :
146   ∀ r s, r < n → s < n → f₂ n r = f₂ n s → r = s := by
147   intro r s hr hs h; simp only [f₂] at h; split_ifs at h <=> omega
148
149 theorem e2_anti (n : ℕ) (hn : n % 6 = 2) :
150   ∀ r s, r < n → s < n → r ≠ s → r + f₂ n r ≠ s + f₂ n s := by
151   intro r s hr hs hrs; simp only [f₂]; split_ifs <=> omega
152
153 /- Needs `8 ≤ n` (`h ≥ 4`): the P1P3 / P2P4 cross cases fail for `h ≤ 3`. -/
154 theorem e2_main (n : ℕ) (hn : n % 6 = 2) (h8 : 8 ≤ n) :
155   ∀ r s, r < n → s < n → r ≠ s → r + f₂ n s ≠ s + f₂ n r := by
156   intro r s hr hs hrs; simp only [f₂]; split_ifs <=> omega
157
158 /- Needs `4 ≤ n` (`h ≥ 2`): at `n = 2` both `f₂ 2 0 = 0` and `f₂ 2 1 = 1`. -/
159 theorem e2_no_fix (n : ℕ) (hn : n % 6 = 2) (h4 : 4 ≤ n) : ∀ r, r < n → f₂ n r ≠ r := by
160   intro r hr; simp only [f₂]; split_ifs <=> omega
161
162 theorem goodEven_f2 (n : ℕ) (hn : n % 6 = 2) (h8 : 8 ≤ n) : GoodEven n (f₂ n) :=
163   (e2_lt n hn, e2_inj n hn, e2_anti n hn, e2_main n hn h8, e2_no_fix n hn (by omega))
164

```

```

165 end EvenLemmas
166
167 /-- Every even board  $n \geq 4$  has a good placement. -/
168 theorem goodEven_exists (n : ℕ) (h4 : 4 ≤ n) (he : n % 2 = 0) : ∃ f, GoodEven n f := by
169   rcases (by omega : n % 6 = 0 ∨ n % 6 = 2 ∨ n % 6 = 4) with h | h | h
170   · exact {_, goodEven_f1 n (Or.inl h)}
171   · exact {_, goodEven_f2 n h (by omega)}
172   · exact {_, goodEven_f1 n (Or.inr h)}
173
174 /-! ## Odd boards: corner extension of the even board  $m$  to board  $m + 1$  -/
175
176 /-- Board  $m + 1$ : rows  $< m$  follow  $g$ , the last row gets the corner column  $m$ . -/
177 def corner (m : ℕ) (g : ℕ → ℕ) (r : ℕ) : ℕ := if r < m then g r else m
178
179 theorem corner_lt (m : ℕ) (g : ℕ → ℕ) (hg : GoodEven m g) :
180   ∀ r, r < m + 1 → corner m g r < m + 1 := by
181   --  $r < m + 1$  is not needed: rows  $< m$  are bounded by  $hg$ , row  $m$  maps to  $m$ .
182   intro r _; obtain ⟨h1, -, -, -⟩ := hg
183   simp only [corner]; split_ifs
184   · have := h1 r < _; omega
185   · omega
186
187 theorem corner_inj (m : ℕ) (g : ℕ → ℕ) (hg : GoodEven m g) :
188   ∀ r s, r < m + 1 → s < m + 1 → corner m g r = corner m g s → r = s := by
189   intro r s hr hs heq; obtain ⟨h1, h2, -, -, -⟩ := hg
190   simp only [corner] at heq; split_ifs at heq
191   · exact h2 r s < _ < _ heq
192   · have := h1 r < _; omega
193   · have := h1 s < _; omega
194   · omega
195
196 theorem corner_anti (m : ℕ) (g : ℕ → ℕ) (hg : GoodEven m g) :
197   ∀ r s, r < m + 1 → s < m + 1 → r ≠ s → r + corner m g r ≠ s + corner m g s := by
198   intro r s hr hs hrs; obtain ⟨h1, -, h3, -, -⟩ := hg
199   simp only [corner]; split_ifs
200   · exact h3 r s < _ < _ hrs
201   · have := h1 r < _; omega
202   · have := h1 s < _; omega
203   · omega
204
205 /-- The corner queen at  $(m, m)$  sits on the main diagonal through  $(r, g r)$ 
206 iff  $g r = r$ ; this is where GoodEven's no-fixed-point clause is used. -/
207 theorem corner_main (m : ℕ) (g : ℕ → ℕ) (hg : GoodEven m g) :
208   ∀ r s, r < m + 1 → s < m + 1 → r ≠ s → r + corner m g s ≠ s + corner m g r := by
209   intro r s hr hs hrs; obtain ⟨-, -, -, h4, h5⟩ := hg
210   -- split_ifs splits on  $s < m$  first here (the goal mentions corner m g s first).
211   simp only [corner]; split_ifs
212   · exact h4 r s < _ < _ hrs
213   · have := h5 s < _; omega
214   · have := h5 r < _; omega
215   · omega
216
217 /-! ## The namesake theorems (statements frozen from the OPE-4 pin) -/
218
219 /-- Optional split: existence for  $n \geq 4$  by the explicit mod 6 construction
220 (Hoffman-Loessi-Moore 1969 / Bernhardsson 1991). The hard direction of the
221 namesake, isolated so it can be landed and reviewed on its own. -/
222 theorem queens_exists_of_four_le (n : ℕ) (hn : 4 ≤ n) :
223   ∃ q : Fin n → Fin n, NonAttacking q := by
224   rcases Nat.even_or_odd' n with ⟨k, rfl | rfl⟩
225   · obtain ⟨f, hf⟩ := goodEven_exists (2 * k) hn (by omega)
226   · exact {_, nonAttacking_of_nat f hf.1 hf.2.1 hf.2.2.1 hf.2.2.2.1}

```

```

227 · obtain (g, hg) := goodEven_exists (2 * k) (by omega) (by omega)
228   exact (_, nonAttacking_of_nat (corner (2 * k) g) (corner_lt _ _ hg) (corner_inj _ _ hg)
229     (corner_anti _ _ hg) (corner_main _ _ hg))
230
231 /-- `n = 2`: no placement (4 functions, kernel `decide`). -/
232 theorem queens_two_none' : ¬ ∃ q : Fin 2 → Fin 2, NonAttacking q := by
233   unfold NonAttacking; decide
234
235 /-- `n = 3`: no placement (27 functions, kernel `decide`). -/
236 theorem queens_three_none' : ¬ ∃ q : Fin 3 → Fin 3, NonAttacking q := by
237   unfold NonAttacking; decide
238
239 /-- Namesake. Existence iff `n ≠ 2 ∧ n ≠ 3` (n = 0 vacuous, n = 1 trivial,
240 n ≥ 4 by explicit construction). -/
241 theorem n_queens_exists_iff (n : ℕ) :
242   (∃ q : Fin n → Fin n, NonAttacking q) ↔ (n ≠ 2 ∧ n ≠ 3) := by
243   constructor
244   · intro hq
245     refine (?, ?)
246     · rintro rfl; exact queens_two_none' hq
247     · rintro rfl; exact queens_three_none' hq
248   · rintro (h2, h3)
249     rcases Nat.lt_or_ge n 4 with h | h
250     · interval_cases n
251       · exact (id, by unfold NonAttacking; decide)
252       · exact (id, by unfold NonAttacking; decide)
253       · exact absurd rfl h2
254       · exact absurd rfl h3
255     · exact queens_exists_of_four_le n h
256
257 /-! ## Sanity checks (decide-level only; no proof search) -/
258
259 /-- The Level A `n = 4` witness `[1, 3, 0, 2]` satisfies the function-encoded
260 predicate. Confirms the encoding agrees with `ProofLab/NQueens.lean` on the
261 one concrete board both files know about. -/
262 example : NonAttacking ![1, 3, 0, 2] := by
263   unfold NonAttacking
264   decide
265
266 /-- `f₁ 4` is the Level A witness `[1, 3, 0, 2]`. -/
267 example : (List.range 4).map (f₁ 4) = [1, 3, 0, 2] := by decide
268
269 /-- `f₂ 8` is Bernhardsson's `n = 8` board `[3, 5, 7, 1, 6, 0, 2, 4]`. -/
270 example : (List.range 8).map (f₂ 8) = [3, 5, 7, 1, 6, 0, 2, 4] := by decide
271
272 end ProofLab.NQueensTheorem

```

B Verification transcripts

B.1 Reviewer's Lean checks

Output of lake env lean review_checks.lean (review_checks.out):

```

'ProofLab.NQueensTheorem.n_queens_exists_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProofLab.NQueensTheorem.queens_exists_of_four_le' depends on axioms: [propext, Quot.sound]
'ProofLab.NQueensTheorem.queens_two_none'' depends on axioms: [propext, Classical.choice, Quot.sound]
'ProofLab.NQueensTheorem.queens_three_none'' depends on axioms: [propext, Classical.choice, Quot.sound]
lake env lean ../../attacks/n-queens-20260912-180329/review_checks.lean 1.69s user 1.59s system 38% cpu 8.543

```

B.2 Attack Lead's brute-force evidence (not proof)

Output of `verify_construction.py 1000` (`verify_output.txt`). Note that this script used signed integers and therefore could not detect the $\mathbb{N}$ -truncation described in Section 5.3.

```
range: 4 <= n <= 1000; boards checked per residue class n%6: {0: 166, 1: 166, 2: 166, 3: 166, 4: 167, 5: 166}
clauses: bound, Function.Injective, i+q i ≠ j+q j, i+q j ≠ j+q i (all ordered pairs)
primary construction (E1 / E2 / odd corner extension): PASS
even-n no-fixed-point side condition: PASS
alt knight (2r % n) on n%6 in {1,5}: PASS
bonus: E2 formula on n%6 = 4: PASS
sample n=8 (E2): [3, 5, 7, 1, 6, 0, 2, 4]
sample n=9 (odd, corner ext of n=8): [3, 5, 7, 1, 6, 0, 2, 4, 8]
sample n=10 (E1): [1, 3, 5, 7, 9, 0, 2, 4, 6, 8]
```

C Ticket timeline

UTC	ticket	role	event
17:37	OPE-2	board	names the problem; approves tooling
17:44	OPE-2	Director	Lean 4.10.0 installed and green; plan and child chain posted
17:51	OPE-3	Scout	dossier, pin, novelty screen (PR #3)
17:53	OPE-2	Director	Gate 1: dossier approved; names frozen
17:58	OPE-4	Formalist	statement stub compiles with sorryAx (PR #4)
18:00	OPE-2	Director	Gate 2: stub approved; Attack Lead released
18:08	OPE-5	Attack Lead	construction, brute check, lemma list (PR #5)
18:12	OPE-2	Director	Gate 4: plan approved; Formalist proof released
18:20	OPE-6	Formalist	both theorems closed, zero sorry (PR #6)
18:23	OPE-2	Director	Gate 6: rebuild and axioms re-checked; Reviewer released
18:28	OPE-7	Reviewer	approve with residuals (PR #7)
18:34	OPE-2	Director	close-out: integration, erratum, ledger, claim packet (PR #8)